CS 200: Lab 8

You may access notes and pre-labs from LMS; Time Given=90 minutes

"By proceeding I certify that I have neither received nor given unpermitted aid on this component and that I have reported all such incidents observed by me in which unpermitted aid is given."

Submit after zipping the assignment folder and renaming it to `'<rollnumber>_Lab8.zip'`. Upload your zip file in the corresponding LMS submission tab. Example, `'27100202_Lab8.zip'`

Note: For this lab, you have been given class definitions in header files, however you will write your code in file 'bankaccount.cpp' file and the 'ewallet.cpp' files. The .cpp files provide you with the required function definitions

1 Introduction to Operator Overloading

Welcome to the lab on **Operator Overloading** in C++!!

As you're working through the lab, some tasks may seem daunting because they use new concepts, but remember that operator overloading is one of the simplest parts of OOP. Every function can be implemented in just a couple of lines. Take a deep breath and let's get you through this!

Operator overloading lets user-defined classes use operators like `+`, `-`, and `==` in the way you might use them for built-in types like `int`. It improves readability and usability. For example:

```
// Overload + operator to add two Complex numbers
    Complex operator+(const Complex& c) {
        return {real + c.real, imaginary + c.imaginary};
    }
//sum = c1 + c2;
```

1.1 Friend Functions

Friend functions in C++ allow external functions to access private and protected members of a class. They are declared with the `friend` keyword inside the class but are not member functions.

1.2 Stream Insertion Operator

The stream insertion operator («) is often overloaded using a friend function to allow objects to be printed with `std::cout`. Since `operator«` is not a member function, it needs access to private members, which is achieved by declaring it as a `friend` inside the class. For example the Complex Number Class:

```
friend ostream& operator<<(ostream& os, const Complex& c) {
    return os << c.real << " + " << c.imag << "i"; }

//cout << c; will print in the form 3 + 4i
```

2 Lab Overview

Now that we've revised the basics, in this lab, you will implement a small banking system featuring two classes: **Wallet** and **BankAccount**. Both classes will make extensive use of operator overloading to perform arithmetic operations, comparisons, and stream output.

2.1 Task 1: The Wallet Class (75 marks)

The **Wallet** class represents a digital wallet with the following features:

Attributes:

- `ownerName`: Name of the wallet owner.
- `balance`: Current balance as `double` .
- `isPremium`: A Boolean indicating whether the wallet is premium (Non-premium wallets cannot exceed a balance limit of Rs. 50,000).

You will need to implement the following methods:

2.1.1 Constructors

- **`Wallet()`:** Default constructor initializing an empty wallet with `""` as owner name, `0` balance, and non-premium status.
- **`Wallet(string owner, double initialBalance, bool premium)`:** Parameterized constructor setting the wallet's owner, balance, and premium status.

2.1.2 Getters

- **`getBalance()`** Returns the wallet's balance.
- **`getOwnerName()`** Returns the wallet owner's name.
- **`isPremiumStatus()`** Returns whether the wallet has premium status.

2.1.3 Operator Overloading

- **`operator+`** Merges two wallets, returning a new wallet with the combined balance and the same owner.

  `Example: Wallet w4 = w2 + w3;`

- **`operator-`** Unlike `operator-=`, `operator-` withdraws money from the wallet and shifts the remaining money to a new sub-wallet. This only happens if enough balance is available; otherwise, returns the same wallet unchanged.

  ```
  Example: Wallet w5 = w2 - 5000; // Deducts from w2, shifts the rest to w5
  ```

- **`operator+=`** Adds money to the wallet, enforcing a Rs. 50,000 balance limit if it's not premium.

  ```
  Example: w2 += 5000; \\adds 5000 to w2
  ```

- **`operator-=`** Deducts money from the wallet if sufficient balance exists. Does not shift money to a new sub-wallet after deducting like it did with `operator-`

  ```
  Example: w2 -= 5000;
  ```

- **Comparison Operators (`==`, `!=`, `<`, `>`):** Compare wallet balances and returns a boolean value

- **Stream Insertion (`operator«`):** Allows `cout « wallet;` to print the wallet's details. Note: Your test cases will not pass unless your format, including spaces, is the exact same.

  ```
  Format: Single Space
  Owner: Maryam Ali | Balance: 15000 | Premium: No
  ```

2.2 Task 2: The BankAccount Class (25 marks)

The **BankAccount** class models a bank account that interacts with e-wallets:

Attributes:

- `accountHolder`: The name of the account holder.
- `accountBalance`: The balance of the current account as `double`.

You will need to implement the following methods:

2.2.1 Constructors

- `Default Constructor:` Creates an empty bank account with no holder (`""`) and a balance of `0`.
- `Parameterized Constructor:` Initializes a bank account with the given account holder name and initial balance.

2.2.2 Getters

- `getBalance():` Returns the current balance of the bank account.
- `getHolderName():` Returns the name of the account holder.

Operator Overloading

- `operator+`: Allows your Bank Account to absorb your wallet, by shifting the money from a `Wallet` to the `BankAccount`, creating a new merged accountwith the updated balance.

  ```
  Example: BankAccount acc2 = acc1 + w1; // New account with increased balance
  ```

- `operator-`: Deducts money from the account, shifting the remaining money to the new account. If there are insufficient funds, it returns the same account.

  ```
  Example: BankAccount acc2 = acc1 - 2000; // New account with decreased balance
  ```

- `operator«`:Allows printing a `BankAccount` object using `cout`. Follow the exact format or your test cases won't pass

  ```
  Format: Single Space
  Account Holder: Maryam Ali | Balance: 50000
  ```

Running and Compiling your Code

```cpp
//To compile
g++ ewallet.cpp bankaccount.cpp test.cpp -o test.exe

//To run
./test.exe

//you may also insert && in between both commands and execute them in one-go
```

Common Mistakes & Debugging Tips

- **Segmentation Faults & Undefined Behavior:**
  - Even if you haven't implemented a method yet, make sure it is returning something (even if it's incorrect/null/empty), otherwise your test cases may not run.
  - Always check for valid conditions (e.g., not withdrawing more than the current balance).
  - When overloading operators, do not modify objects that should remain unchanged (especially in non-assignment operators).
- **Forgetting to Check Limits:**
  Non-premium wallets may have restrictions (e.g., a deposit that would exceed a limit should be rejected) and a and an account cannot have balance in the negative. Ensure you read the manual carefully for such restrictions.

- **Operator Return Types:**
  Make sure that arithmetic operators (`+`, `-`) return new objects while assignment operators (`+=`, `-=`) modify the current object and return a reference.
- **Stream Insertion Operator:**
  Ensure the output format is exact (including spacing and punctuation), as the test cases compare strings.
- **Debugging:**
  - Use simple `cout` statements to trace the values in your functions if your output is not as expected.
  - Verify that your constructors are initializing all members.

Good luck, you got this!